institut montilivi

EXERCICIS

PRÀCTICA RA1 A3

CONTINGUT

01 1. Conceptes prèvis, estructura i objectius de la pràctica.

02 1. Primera part: implementació amb `yield return`

03 2. Segona part: implementació d'un enumerador personalitzat

04 3. Tercera part: consultes LINQ sobre les incidències

05 Requisits generals de lliurament

MÒDUL

ACCÉS A DADES

LLIURA-HO A CLASSROOM50 ↗

1. CONCEPTES PRÈVIS, ESTRUCTURA I OBJECTIUS DE LA PRÀCTICA.

- **IMPLEMENTANT UN ENNUMERADOR SOBRE UN STREAMREADER**

En aquesta pràctica repassarem conceptes de programació en C# que ja hem treballat durant el primer curs, especialment les **interfícies** IEnumerable, **enumeradors**, els **fitxers de text** i la **programació amb coİleccions** LinQ.

Aplicarem aquests conceptes a un escenari pràctic d'accés a dades mitjançant fitxers de text amb format CSV. Treballarem amb un fitxer que conté un conjunt d'incidències informàtiques d'una empresa i desenvoluparem consultes per filtrar, seleccionar, ordenar i agrupar aquestes dades.

L'objectiu principal és aprendre a tractar les dades d'un fitxer com una seqüència enumerable, de manera que puguem utilitzar `foreach` i les operacions de **LINQ** sense haver de carregar necessàriament tot el contingut del fitxer en memòria.

Per assolir aquest objectiu, desenvoluparem dues implementacions diferents d'un enumerador:

1. Una primera implementació de `StreamReaderEnumerable` que utilitzi `yield return` per generar els elements de la seqüència.
2. Una segona implementació que utilitzi un enumerador personalitzat i implementi explícitament els mecanismes de recorregut: `MoveNext()`, `Current`, `Reset()` i `Dispose()`.

Finalment, aplicarem consultes LINQ a les dades obtingudes mitjançant aquestes dues implementacions i compararem el seu funcionament.

. INTRODUCCIÓ TEÒRICA: `IENUMERABLE<T>` I `IENUMERATOR<T>`

En C#, una interfície defineix un contracte que una classe ha de complir per poder oferir un determinat comportament.

La interfície genèrica `IEnumerable<T>` representa una seqüència d'elements d'un tipus determinat. En el nostre cas, com que cada línia del fitxer és una cadena de text, treballarem amb `IEnumerable<string>`.

Aquesta interfície permet obtenir un enumerador mitjançant el mètode `GetEnumerator()`. L'enumerador és l'objecte encarregat de recórrer la seqüència element a element.

Cal distingir aquests dos conceptes:

- `IEnumerable<T>`: representa una seqüència i permet obtenir-ne un enumerador.
- `IEnumerator<T>`: permet avançar per la seqüència i consultar l'element actual.

El bucle `foreach` pot recorrer una seqüència `IEnumerable<T>` cadascun dels elements sense que haguem de gestionar manualment l'avanç de l'enumerador `IEnumerator<T>`.

• LA INTERFÍCIE `IENUMERABLE<STRING>`

La classe que desenvoluparem, `StreamReaderEnumerable`, haurà d'implementar `IEnumerable<string>`.

Això ens permetrà utilitzar objectes d'aquesta classe amb `foreach`, però també amb les operacions de LINQ, com ara `Where()`, `Select()`, `Count()`, `OrderBy()` i `GroupBy()`, ...

Per exemple:

CODI C#

```
var dades = new StreamReaderEnumerable("LogIncidencies.csv");

foreach (string linia in dades)
{
    Console.WriteLine(linia);
}
```

Perquè aquest codi funcioni, la classe haurà de proporcionar una implementació de `GetEnumerator()`.

• LA INTERFÍCIE `IENUMERATOR<STRING>`

Un enumerador genèric implementa, entre altres membres, els següents:

- `MoveNext()`: avança fins al següent element i retorna `true` si existeix.
- `Current`: proporciona l'element actual.
- `Reset()`: intenta reiniciar el recorregut. En alguns enumeradors aquesta operació no està suportada.
- `Dispose()`: allibera els recursos que utilitza l'enumerador.

Quan utilitzem `yield return`, el compilador genera automàticament bona part de la implementació de l'enumerador.

En canvi, quan implementem `IEnumerator<string>` manualment estem implementant un cursor personalitzat, per tant, hem de gestionar nosaltres mateixos aquests membres i l'estat del recorregut.

• PER QUÈ UTILITZAREM `STREAMREADER`?

La classe `StreamReader` permet llegir un fitxer de text de manera seqüencial. Amb el mètode `ReadLine()` podem llegir-ne una línia cada vegada.

Si encapsulem aquesta lectura en un enumerable, podrem tractar el fitxer com una seqüència de cadenes de text i aplicar-hi les operacions de `LINQ`.

És important recordar que un `StreamReader` és un recurs que s'ha de tancar correctament. Per això, haurem de dissenyar adequadament la gestió dels recursos i el cicle de vida de l'enumerador.

. FITXER DE DADES: `LOGINCIDENCIES.CSV`

Treballarem amb un fitxer que conté 40 incidències fictícies d'una empresa.

Cada línia representa una incidència i els camps estan separats pel caràcter punt i coma (`;`). El fitxer conté sis camps:

ID	DATA	DEPARTAMENT	PRIORITAT	ESTAT	DESCRIPCIÓ
`INC-1001`	`2026-09-01`	`Sistemes`	`Alta`	`Resol incidència`	`Servidor principal reiniciat després d'una caiguda inesperada`

Descarregueu o copieu el fitxer proporcionat amb la pràctica i deseu-lo com a `LogIncidencies.csv` al directori de treball del programa.

Aquesta vegada no hi ha complicacions a l'hora de separar els camps :). Utilitzarem un format **CSV simplificat**, sense camps que continguin separadors escapats ni comes internes a un camp. En un CSV real amb aquestes característiques, caldria utilitzar un analitzador CSV adequat.

---

1. PRIMERA PART: IMPLEMENTACIÓ AMB `YIELD RETURN`

En aquesta primera part implementarem la classe `StreamReaderEnumerable`, que representarà el contingut del fitxer com una seqüència de cadenes de text.

REQUISITS

1. Creeu una classe anomenada `StreamReaderEnumerable`.
2. Feu que implementi `IEnumerable<string>`.
3. Afegiu un constructor que rebi la ruta del fitxer.
4. Implementeu `GetEnumerator()` utilitzant `yield return` per retornar les línies del fitxer progressivament.
5. Implementeu també el mètode heretat de la interfície no genèrica `IEnumerable`.

La lectura haurà de fer-se línia a línia, fins que `ReadLine()` indiqui que s'ha arribat al final del fitxer.

Quan utilitzeu `yield return`, recordeu que el codi de l'iterador s'executa progressivament a mesura que es recorren els elements. La gestió del `StreamReader` haurà de garantir que el fitxer es tanqui quan s'acabi el recorregut o quan aquest s'interrompi.

COMPROVACIÓ

Creeu un programa principal que instanciï `StreamReaderEnumerable` i mostri totes les línies mitjançant un `foreach`.

Comproveu que:

- Es llegeixen totes les línies del fitxer.
- Cada línia es retorna una sola vegada en cada recorregut.
- El fitxer es tanca correctament quan finalitza l'enumeració.

2. SEGONA PART: IMPLEMENTACIÓ D'UN ENUMERADOR PERSONALITZAT

En aquesta segona part repetirem el mateix exercici, però sense utilitzar `yield return`.

L'objectiu és comprendre com funciona internament el mecanisme d'enumeració de C#.

Per fer-ho, separarem la implementació en dues classes:

- `StreamReaderEnumerable`, que implementarà `IEnumerable<string>` i crearà enumeradors.
- `StreamReaderEnumerator`, que implementarà `IEnumerator<string>` i gestionarà el recorregut del fitxer.

REQUISITS DE `STREAMREADERENUMERABLE`

La classe haurà de conservar la ruta del fitxer i implementar `GetEnumerator()`, que crearà i retornarà un objecte `StreamReaderEnumerator`.

També haurà d'implementar el mètode de la interfície no genèrica `IEnumerable`.

REQUISITS DE `STREAMREADERENUMERATOR`

Aquesta classe haurà d'implementar els membres següents:

`MoveNext()`

Ha de llegir la línia següent i retornar `true` quan hi hagi un element disponible. Quan s'arribi al final del fitxer, haurà de retornar `false`.

`Current`

Ha de retornar la línia actual. Cal controlar que no s'intenti obtenir l'element actual quan l'enumerador encara no ha començat o quan ja s'ha acabat el recorregut.

`Reset()`

Posem el current a l'inici del fitxer (null). Alliberem memoria del buffer i reiniciem el punter del fitxer al principi.

- `sR.discardBufferedData();`
- `sR.BaseStream.Seek(0, SeekOrigin.Begin);`
- `current = null;`

`Dispose()`

Ha de tancar el `StreamReader` i alliberar els recursos. Cal evitar que el mateix recurs es tanqui repetidament de manera incorrecta.

Tingueu en compte que `IEnumerator<string>` també hereta de la interfície no genèrica `IEnumerator`. Per això, la implementació manual haurà d'incloure la propietat `object IEnumerator.Current`, que pot delegar en la propietat genèrica `Current`.

COMPROVACIÓ

Torneu a executar el programa principal i comproveu que podeu recórrer el fitxer amb `foreach`, exactament igual que en la primera part.

La diferència és que, en aquesta ocasió, haureu implementat manualment el mecanisme de recorregut.

3. TERCERA PART: CONSULTES LINQ SOBRE LES INCIDÈNCIES

Ara que disposem d'una classe enumerable, utilitzarem LINQ per consultar les dades.

Afegiu al projecte:

CODI C#

```
using System.Linq;
```

Les consultes s'han de fer a partir de `StreamReaderEnumerable`. No es permet utilitzar `File.ReadAllLines()` ni carregar prèviament tot el fitxer en una llista per fer les consultes.

Totes les consultes han de partir de la seqüència proporcionada per l'enumerador i utilitzar les operacions LINQ corresponents.

1. Comptar les incidències

- Mostreu el nombre total de línies del fitxer utilitzant LINQ, sense fer un `foreach` explícit per comptar-les.
  - Concepte principal: `Count()`.

2. Cercar incidències relacionades amb servidors

- Mostreu totes les línies que continguin la paraula «servidor», sense distingir entre majúscules i minúscules.
- Conserveu el contingut original de les línies en mostrar els resultats.
  - Conceptes principals: `Where()`, `Contains()` i `StringComparison`.

3. Filtrar les incidències amb prioritat crítica

- Mostreu totes les incidències la prioritat de les quals sigui «Crítica».
- Heu d'extreure el camp de prioritat de cada registre i comparar-lo amb el valor corresponent.

- Conceptes principals: `Where()`, `Split()` i accés als camps.

4. Seleccionar les incidències pendents

- Mostreu totes les línies que tinguin l'estat «Pendent».
- La comparació s'ha de fer sobre el camp de l'estat, no sobre el text complet de la línia.
  - Conceptes principals: `Where()` i indexació dels camps.

5. Ordenar les incidències alfabèticament

- Mostreu totes les incidències ordenades pel nom del departament i, dins de cada departament, per data.
  - Conceptes principals: `OrderBy()` i `ThenBy()`.

6. Mostrar una part de la seqüència

- Mostreu les incidències de la tercera a la cinquena posició, amb totes les lletres en minúscula.
- Considereu que la primera posició és la número 1.
  - Conceptes principals: `Skip()`, `Take()` i `Select()`.

7. Obtenir els departaments sense repeticions

- Mostreu els noms dels departaments que apareixen al fitxer, sense duplicats i ordenats alfabèticament.
  - Conceptes principals: `Select()`, `Distinct()` i `OrderBy()`.

8. Comptar les incidències per estat

- Mostreu quantes incidències hi ha en cada estat: «Pendent», «En curs» i «Resol incidència».
- El resultat ha d'indicar el nom de cada estat i el nombre d'incidències corresponent.
  - Conceptes principals: `GroupBy()`, `Select()` i `Count()`.

9. Identificar el departament amb més incidències

- Determineu quin departament acumula més incidències i mostreu-ne el nom i el nombre de registres.
  - Conceptes principals: `GroupBy()`, `Select()`, `OrderByDescending()` i `First()`.

10. Crear un resum de les incidències crítiques pendents

- Mostreu l'identificador i la descripció de totes les incidències que compleixin simultàniament aquestes dues condicions:
  - La prioritat és «Crítica».
  - L'estat és «Pendent».
- Els resultats s'han de mostrar en ordre cronològic.
  - Conceptes principals: `Where()`, `Select()` i `OrderBy()`.
- Per ordenar cronològicament, utilitzeu el camp de la data. Com que les dates estan escrites en format `yyyy-MM-dd`, l'ordenació alfabètica d'aquest camp també respecta l'ordre cronològic.

REQUISITS GENERALS DE LLIURAMENT

Cal lliurar el projecte de C# amb les dues implementacions i les consultes resoltes al github classroom. Recordeu fer un bon ús dels `commits` i dels `messages` de `commit` per poder fer un seguiment de l'evolució del projecte.

El projecte ha d'incloure:

1. La classe `StreamReaderEnumerable` amb la implementació basada en `yield return`.
2. La classe `StreamReaderEnumerable` i la classe `StreamReaderEnumerator` amb la implementació manual.
3. Un programa principal que permeti provar les dues alternatives.
4. Les deu consultes LINQ resoltes i identificades clarament.
5. Una breu explicació dels avantatges i inconvenients de les dues implementacions.

Podeu organitzar les dues alternatives en projectes diferents o bé mantenir-les en fitxers diferents, sempre que no hi hagi conflictes de noms.

Resultat esperat: en acabar la pràctica, hauríeu de ser capaços d'implementar una seqüència enumerable a partir d'un fitxer, entendre el funcionament dels enumeradors i aplicar LINQ per consultar les dades de manera clara i expressiva.

LLIURAMENT

Lliura-ho a Classroom50: https://classroom50.org/Montilivi-DAM2/acces-dades/assignments/activitat-a3/submissions